# Day 9 Capstone Brief

Issued at stand-up · Embargo: do not open before stand-up

## BUSINESS CONTEXT

HBP operates a growing e-commerce business selling consumer electronics and accessories across the UK, US, Australia, and Canada. The data engineering team has spent eight days building a production ELT pipeline tracking customer orders, product performance, and revenue by geography.

HBP's commercial leadership has raised a second question: **which customers are at risk of churning due to unresolved support issues?** The customer success team has no visibility into how ticket volume and resolution speed relate to customer spend. Today you will close that gap by extending the existing pipeline.

## YOUR OBJECTIVE

Ingest and model a new dataset — customer support tickets — and join it with order history already in the warehouse. The commercial team needs a mart model that answers: *which HBP customers have both high spend and unresolved support issues?*

## DATASET: CAPSTONE_SUPPORT_TICKETS.CSV

You will receive one new CSV file with 12 rows of HBP support ticket data. The file has not been cleaned. Inspect it carefully before writing any code.

| Column | Description |
|---|---|
| ticket_id | Unique identifier — format T001, T002, etc. |
| customer_id | Foreign key — links to existing customers |
| subject | Free-text description of the issue |
| category | Billing · Delivery · Product · Account |
| priority | Ticket priority: high, medium, or low |
| status | Current state: open, resolved, or closed |
| created_at | Date the ticket was raised |
| resolved_at | Resolution date — NULL for open tickets |
| agent_name | Support agent who handled the ticket |

**Known data quality issues exist in this file. Identify them, document them in Notion before writing any code, and handle them in your ingestion script.**

## WHAT YOU MUST BUILD

**Pre-coding analysis** — Document quality issues, primary key, foreign key link, and business question before any code.

**ingest_capstone.py** — Four-function pattern. Handle NULL resolved_at gracefully. Compute days_to_resolve.

**SUPPORT_TICKETS raw table** — Create in HBP_RAW.INGEST before running the script. All columns VARCHAR.

**stg_support_tickets.sql** — dbt staging via {{ source() }}. CTE pattern. accepted_values test on status.

**int_ticket_metrics.sql** — One row per customer_id: ticket counts by status, high_priority_tickets, avg_days_to_resolve.

**mart_support_summary.sql** — Join int_ticket_metrics with mart_customer_summary. Add customer_health CASE column. All 8 customers must appear.

**schema.yml tests** — unique/not_null on customer_id, relationships test to stg_customers, accepted_values on customer_health.

**dbt docs + presentation** — Lineage DAG screenshot. 5 slides: architecture, data quality, DAG, live query + interpretation, next steps.

## SUCCESS CRITERIA

✔ dbt run: 8 of 8 models OK
✔ dbt test passes incl. relationships test on mart_support_summary
✔ customer_health: no NULL values — all 8 customers classified
✔ Lineage DAG shows mart_support_summary connected to both upstream models
✔ Slide 4 live query result interpreted for a non-technical stakeholder
✔ GitHub commit with all new files before EOD

## DAY 10 — LIVE PRESENTATION

You present your 5-slide deck live to your trainer and fellow. Be ready to explain every decision: how you defined customer_health, how the mart connects to the existing pipeline, and what you would build next. Expect direct questions.

## TECHNOLOGY STACK

| | |
|---|---|
| **Ingestion** | Python (pandas) — ingest_capstone.py |
| **Raw layer** | Snowflake — HBP_RAW.INGEST |
| **Staging** | dbt view — HBP_ANALYTICS.STAGING |
| **Intermediate** | dbt view — HBP_ANALYTICS.INTERMEDIATE |
| **Mart** | dbt table — HBP_ANALYTICS.MARTS |
| **Version control** | Git — commit all files before EOD |